Plausible Fiction on MeTTaIL

A Requirements & Design Document for a Typed Marketplace of Programmatic Intentions

F1R3FLY.io · in dialogue with D. Spivak (Topos Institute)

June 30, 2026 — Draft for discussion

Abstract

Plausible Fiction (PF) is a marketplace in which a participant publishes a *programmatic* description of a good or service they want produced — "plausible" because it is a runnable artifact, "fiction" because it contains *holes* that no one party can yet fill. Following Spivak's original framing, a PF is a term in a dependently typed lambda calculus; a PF with holes is a *context*. We refine that framing by distinguishing holes that must be filled by *other agents* in the market from holes that are filled by *ordinary computation*. This document specifies the system in two layers on top of MeTTaIL (rholang 1.4): (i) an object language — a dependently typed λ-calculus with typed holes — given as a MeTTaIL `language!` definition, whose terms ride on rho channels; and (ii) a coordination protocol written in MeTTaIL's built-in embedded rholang that manages the full PF lifecycle: publish, match, broker, ground, run, monitor, and either promote to a *template* on success or *retire* (hide or advertise) on failure. Because a PF can branch on outcomes, its open holes are not a flat list but a *conditional demand tree* (CDT) — a finite dependent polynomial fibering the holes owed over the outcomes not yet observed; we make `holes(.)` return this object and show how the coordinator arms its immediate frontier while subscribing to its branch edges. We give code illustrations throughout, a lifecycle state machine, a numbered requirements catalogue, a worked end-to-end example, and an honest accounting of what is established versus conjectural.

Contents

1 Purpose and scope

This document specifies *what* the Plausible Fiction service must do and *how* it maps onto MeTTaIL, at a level of detail sufficient to begin implementation and to anchor continued design discussion with the Topos Institute. It is deliberately concrete about the MeTTaIL surface: the object language is given as a `language!` specification, and every coordination behaviour is given as embedded rholang. It is deliberately honest about its open problems (§10): several matching and templating questions are undecidable in full generality, and the design's central move is to *delegate* those problems to the market rather than to a solver.

Non-goals. We do not re-specify rholang itself. MeTTaIL ships an embedded rholang as its coordination language, and separately offers a language-definition facility (it can even give a meta-circular account of rho); the latter is what we use to define the object language, and the former is what we use to coordinate. We treat both as given.

2 Conceptual model

2.1 A PF is a typed context

Fix a dependently typed λ-calculus with a sort Term of terms (which, being dependent, also serves as the sort of types). A *Plausible Fiction* is a Term that may contain *typed holes*. Writing $?h{:}T$ for a hole named h with expected type T:

$$\mathsf{PF} \quad ::= \quad t \quad \text{where} \quad t \text{ ranges over } \mathsf{Term} \text{ with holes } ?h{:}T.$$

A hole is a typed metavariable; a term with holes is exactly a *many-holed context*. The two defining slogans become precise:

- **Plausible** = the description is a real, type-checkable program fragment, not prose.
- **Fiction** = it has holes; it asserts the *existence* of fillers it does not yet possess.

2.2 Grounding and running

A PF is *grounded* when it has no *agent* holes (defined next). A grounded PF is a closed program that can be *run*: every party who contributed a sub-PF performs their part, in the world, under coordination and monitoring. "Filling a hole" is plugging another PF into it; when the last agent hole is plugged the PF becomes grounded and is eligible to run.

2.3 Two kinds of hole

This is our refinement of Spivak's model, and it drives the whole lifecycle.

Agent holes $?h{:}T$.
Filled by *another PF supplied by another participant*. These are market demands. They are the only holes that the lifecycle advertises, brokers, and waits on. grounded counts *only* agent holes. An agent hole *resolves* in one of two ways, which affects only *when*, not *how*, it enters the market (§2.4): *filler-resolved* (the default — a market plug supplies it), or *outcome-resolved* (an *observation* hole whose value is revealed only by running a step, via attestation).

Computation holes.
Filled by *standard computation*: β/η-reduction, normalization, and elaboration of implicit arguments (ordinary metavariables solved by unification). These never become market demands; they are discharged by the evaluator/elaborator.

Concretely: a redex such as $(\lambda(x{:}A).b)\,a$ is a computation hole — it is "filled" by β. An implicit argument that the elaborator can infer is a computation hole. A slot for "a licensed electrician who will wire the venue" is an agent hole — no amount of computation produces the electrician; *another participant* must offer one.

2.4 Branch-dependent holes and the demand polynomial

A PF does not merely *have* holes; it *branches*, and which holes are owed can depend on an outcome that has not landed yet. Spivak's minimal case: Alice books a venue before she knows whether it will be indoor or outdoor; an outdoor booking owes a rain contingency (a tent), an indoor one must *not*.

$$\texttt{case}\ v\ \texttt{of}\ \{\ \mathsf{Indoor}\ v_{\text{in}} \Rightarrow \mathit{book}\, v_{\text{in}}\, c \mid \mathsf{Outdoor}\ v_{\text{out}} \Rightarrow ?h_{\text{Tent}} : \mathsf{Tent}(v_{\text{out}});\ \ \mathit{book}\, v_{\text{out}}\, c\, t\ \}$$

Here h_{Cater} is owed on *every* branch, while h_{Tent} is owed *iff* the venue v resolves $\mathsf{Outdoor}$ — and its type $\mathsf{Tent}(v_{\text{out}})$ *depends on the outcome's payload*. So the open-hole set is a *function of* v, and v is not yet present when we ask for the holes.

A flat `List of [h, ty]` cannot represent this: it is wrong on one branch (it either spuriously demands a tent indoors, or omits the tent contingency outdoors). The two naive escapes both lose something essential:

Eager (hoist h_{Tent} up front).
A tent is then demanded, brokered, and waited on even for indoor venues — an obligation that should not exist.

Lazy (let the venue filler introduce h_{Tent} on republish).
Operationally fine, but then *no PFLam term ever says "outdoor owes a tent"*: the contingency exists only after a filler commits, so the outdoor branch cannot be reasoned about, priced, or checked before grounding.

The branch is already *in the term*, so we take neither escape. We make `holes(.)` return a *conditional demand tree* (CDT): a finite tree whose nodes carry a *frontier* of holes armed immediately and in parallel, and whose edges are *outcomes of a scrutinee* under which a sub-CDT continues. This is a *dependent polynomial* (an interaction structure): positions are outcome-states, the directions at a position are the holes owed there. h_{Cater} lands in the root frontier; h_{Tent} lives in the fiber over $v = \mathsf{Outdoor}$, with its type binding v_{out}. Section 4.4 makes this precise and shows it is computed — not annotated — from the term Alice already wrote.

2.5 The market is the search procedure

A subtle but load-bearing observation. "Types mediate matching" means a candidate filler q fits a hole $?h{:}T$ iff q *checks* at type T in the hole's local context. *Checking a given q against T is decidable* (bidirectional type-checking, assuming normalization). *Finding* such a q is higher-order search and is undecidable in general. The PF design's answer is not to run a solver: it is to publish the demand and let other agents *offer* candidates, which the type guard merely *checks*. The rho market — forwarders, brokers, offers — is the (distributed, human-and-machine, economically incentivized) search procedure; the type system is only the acceptance test. See §10 for the formal consequences.

3 Architecture

Three strata (Figure 1):

1. **Object language `PFLam`** (§4): a dependently typed λ-calculus with typed holes, defined with MeTTaIL's `language!`. Pure, total operations: bidirectional typing, holes$(\cdot)$, grounded$(\cdot)$, and $\triangleleft$.
2. **Bridge** (§5): a small set of rho-callable predicates/operators — `pf= ty|`, *`grounded`*`(pf)`, *`plug`*`(pf,h,q)`, *`holes`*`(pf)` (now returning a CDT), and `resolve(pf,s)` — that lift `PFLam` operations into embedded rholang as `Bool`/payload/`Name` folds. This is the only place the two layers touch.
3. **Coordination** (§6): embedded rholang protocols implementing the lifecycle over per-user channel namespaces.

4 Object language `PFLam`

We give `PFLam` as a Pure-Type-System–style dependent calculus with a single sort Term, a cumulative universe hierarchy, dependent function (Π) and pair (Σ) types (records-as-Σ model goods/services), finite dependent *variants* with their eliminator (`case`, for outcomes and branching — the Option A addition), and the distinguished HOLE constructor (with OBS for outcome-resolved holes). The surface syntax is ASCII to match MeTTaIL's existing examples (`lam x.x`, `(f, x)`).

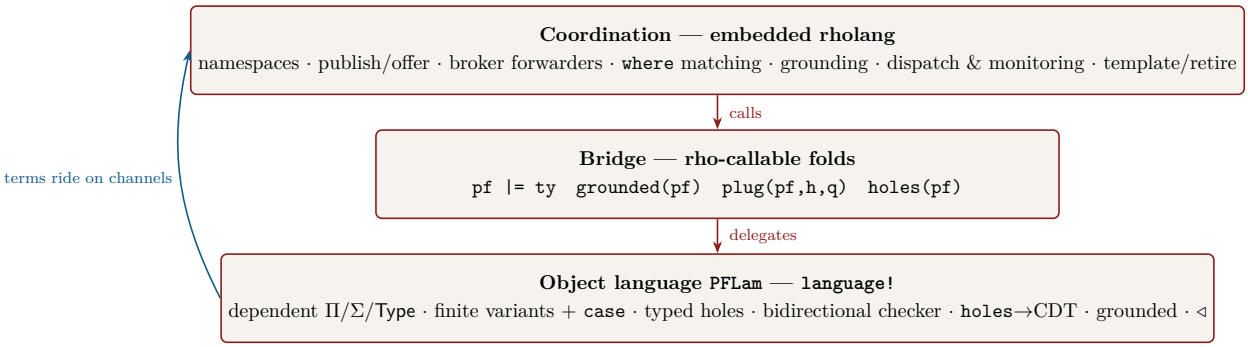

Figure 1: The three strata. Object-language terms are payloads carried on rho channels; the bridge is the sole coupling point.

4.1 Hole interfaces are telescopes

A hole does not float free: it sits under binders and may use the variables in scope at its position. Its *interface* is therefore a *telescope* Γ_h (the local context) together with an expected type T_h. We encode this by Π-closing: a hole records the single closed type $\Pi\Gamma_h.\,T_h$, and a filler is a closed term of that type, *applied* to the local variables at the plug site. This is exactly why Spivak's hole-filling reads as application: plugging `alicePF` at hole h with q is, up to bookkeeping, `alicePF` with q substituted for $?h{:}\Pi\Gamma_h.T_h$ and applied to Γ_h. The minimal spec below keeps HOLE binary (h, T) with the telescope folded into T via Π.

4.2 The `language!` specification

```
language! {
    name: PFLam,

    types {
        Term                        // dependent: terms and types share one sort
        ![String] as HoleId         // agent-hole identifiers (market-visible)
        ![String] as MetaId         // computation-hole / elaboration metavars
        ![String] as Tag            // variant labels (outcome constructors)
        ![u64]    as Lvl            // universe levels
        VariantAlts                 // sequence of  (Tag : Term)   alternatives
        CaseBranch                  // one  (Tag x => body)  arm
        CaseBranches                // sequence of CaseBranch
    },

    terms {
        // ---- core dependent calculus ------------------------------------
        Univ . l:Lvl                        |- "Type" l            : Term;
        Var  . x:Term                       |- x                   : Term;  // bound via HOAS below

        Pi   . dom:Term, ^x.cod:[Term → Term]
             |- "Pi" "(" x ":" dom ")" "." cod                     : Term;
        Lam  . dom:Term, ^x.body:[Term → Term]
             |- "lam" "(" x ":" dom ")" "." body                   : Term;
        App  . fun:Term, arg:Term           |- "(" fun " " arg ")" : Term;

        // dependent records model goods/services as Sigma-bundles
        Sig  . fst:Term, ^x.snd:[Term → Term]
```

```
            |- "Sig" "(" x ":" fst ")" "." snd                  : Term;
        Pair . a:Term, b:Term             |- "<" a "," b ">"     : Term;
        Fst  . p:Term                     |- "fst" p             : Term;
        Snd  . p:Term                     |- "snd" p             : Term;

        // ---- finite (dependent) variants: model outcomes / branching -----
        // A finite labelled sum  < l_1 : A_1 | ... | l_n : A_n >  (Spivak's O_venue).
        Variant . alts:VariantAlts        |- "<" alts ">"        : Term;
        Alt     . lbl:Tag, ty:Term        |- lbl ":" ty          : VariantAlts;  // seq-folded
        Inj     . lbl:Tag, val:Term       |- "inj" lbl val       : Term;
        // Dependent eliminator: motive mot, one arm per label; the arm binds the payload.
        Case    . scrut:Term, mot:Term, brs:CaseBranches
                |- "case" scrut "of" "{" brs "}"                  : Term;
        CaseBr  . lbl:Tag, ^x.body:[Term → Term]
                |- lbl " " x "=>" body                            : CaseBranch;

        // ---- THE distinguishing constructors ----------------------------
        // Agent hole (filler-resolved): supplied by another participant's PF.
        // ty is the Pi-closed interface  Pi Gamma_h . T_h  (see sec. 4.1).
        Hole . h:HoleId, ty:Term          |- "?" h ":" ty        : Term;

        // Observation hole (outcome-resolved): an agent hole whose value is
        // revealed by RUNNING a step (attestation), not by a market plug. It is
        // a legitimate case-scrutinee and a CDT branch point (sec. 4.4, 6.x).
        Obs  . h:HoleId, ty:Term          |- "observe" h ":" ty  : Term;

        // Computation hole: an elaboration metavariable, solved by unification.
        // NEVER advertised to the market; discharged by the elaborator.
        Meta . m:MetaId, ty:Term          |- "%" m ":" ty        : Term;

        // ---- pure operations exposed to the bridge ----------------------
        // grounded(p): the REALIZED path carries no agent holes. Reduction now
        // includes case-reduction (CaseRw), so once a scrutinee hole resolves
        // its branch is taken and only that branch's holes remain (sec. 4.4).
        Grounded . p:Term  |- "grounded" "(" p ")" : Term ![{
            crate::pflam::no_agent_holes(&p)        // → Term::BoolLit(_)
        }] fold;

        // holes(p): the open agent demands as a CONDITIONAL DEMAND TREE (sec 4.4)
        // -- a finite dependent polynomial: a now-frontier of holes armed in
        // parallel, plus outcome-indexed edges over unresolved scrutinees.
        // NOT a flat list: the owed-set is a function of outcomes not yet seen.
        HolesOf  . p:Term  |- "holes" "(" p ")"    : Term ![{
            crate::pflam::demand_tree(&p)           // → CDT (see grammar, sec 4.4)
        }] fold;

        // plug(p, h, q): fill agent hole h in p with filler q (capture-avoiding;
        // q is the Pi-closed filler, re-applied to the local telescope).
        Plug . p:Term, h:HoleId, q:Term
             |- "plug" "(" p "," h "," q ")"        : Term ![{
            crate::pflam::plug_hole(&p, &h, &q)
        }] fold;
    },

    equations {
        // beta / eta / projection are CONVERSION (the equational theory used by
        // type conversion and by the matching test |=).
        Beta . |- ( (lam (x : A) . b) a ) = (subst b x a) ;
        Eta  . |- (lam (x : A) . (f x)) = f ;          // x not free in f
```

```
        SigB1 . |- (fst < a , b >) = a ;
        SigB2 . |- (snd < a , b >) = b ;
        // variant beta: case of a known injection selects and substitutes its arm.
        CaseB . |- (case (inj l a) mot { ... ; l x => body ; ... }) = (subst body x a) ;
    },

    rewrites {
        // directed reduction = "filling computation holes by standard computation"
        BetaRw . |- ( (lam (x : A) . b) a ) ~> (subst b x a) ;
        Fst1   . |- (fst < a , b >) ~> a ;
        Snd1   . |- (snd < a , b >) ~> b ;
        // CASE on a KNOWN injection reduces (computation). On a NEUTRAL scrutinee
        // -- one stuck on an unresolved Hole/Obs -- it does NOT reduce: it is a
        // branch point of the demand tree (sec. 4.4).
        CaseRw . |- (case (inj l a) mot brs) ~> (selectArm brs l a) ;
        // plug reduces structurally once its filler is present; if it resolves a
        // scrutinee, the enclosing CaseRw then fires, realizing a branch.
        PlugRw . |- (plug p h q) ~> (subst_hole p h q) ;
        // congruences (elided): AppCongL/R, PiCong, LamCong, SigCong, PairCong,
        //                       CaseCongScrut, CaseCongArm, ...
    },

    logic {
        // Bidirectional typing as an ascent relation (sketch, sec. 4.3).
        // check(ctx, term, ty)  and  synth(ctx, term, ty).
        // matching test:  models(q, ty)  iff  exists A. synth([], q, A), sub(A, ty).
        relation check(Ctx, Term, Term);
        relation synth(Ctx, Term, Term);
        relation sub(Term, Term);                  // conversion + universe cumulativity
        relation models(Term, Term);
        models(q, ty) <-- synth(ctx0(), q, a), sub(a, ty);
    },
}
```

4.3 Bidirectional typing and the matching relation

The equational theory ($\beta\eta$, projections) is *conversion*, written $\equiv$. Subtyping $<:$ is conversion plus universe cumulativity (and, optionally, width/depth subtyping on Σ-records, which is exactly what lets a richer offer satisfy a leaner demand). The judgments are the usual bidirectional pair; the rules that matter for matching are:

$$\textsc{Hole-Synth}\ \frac{}{\Gamma \vdash ?h{:}T \Rightarrow T} \qquad \textsc{Sub}\ \frac{\Gamma \vdash t \Rightarrow A \qquad A <: B}{\Gamma \vdash t \Leftarrow B} \qquad \textsc{Pi}\ \frac{\Gamma \vdash A \Leftarrow \mathsf{Type}_i \qquad \Gamma, x{:}A \vdash B \Leftarrow \mathsf{Type}_i}{\Gamma \vdash \Pi(x{:}A).B \Rightarrow \mathsf{Type}_i}$$

$$\textsc{Lam}\ \frac{\Gamma, x{:}A \vdash b \Leftarrow B}{\Gamma \vdash \lambda(x{:}A).b \Leftarrow \Pi(x{:}A).B} \qquad \textsc{App}\ \frac{\Gamma \vdash f \Rightarrow \Pi(x{:}A).B \qquad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash f\, a \Rightarrow B[a/x]}$$

Matching. A candidate filler q *satisfies* a hole $?h : T$ — written $q \models T$ — iff $\Gamma_h \vdash q \Leftarrow T$, equivalently (SUB) iff q synthesizes some A with $A <: T$. This is the predicate the rho `where` guard evaluates. It is decidable on the chosen normalizing fragment; the search for q is *not* performed here (§2.5).

Plugging, precisely. For $q \models T_h$,

$$p \triangleleft_h q \;:=\; \texttt{subst\_hole}(p, h, q),$$

the capture-avoiding replacement of every $?h{:}T_h$ in p by q (with q applied to the local telescope Γ_h per §4.1). *Crucially, q may itself contain agent holes.* On the *flat* fragment (no branching) this gives the hole-set recurrence $\text{holes}(p \triangleleft_h q) = (\text{holes}(p) \setminus \{h\}) \cup \text{holes}(q)$, the recursive engine of the marketplace: filling a hole can introduce new holes, deferred to new participants. With branching this becomes a splice on the demand *tree* rather than the set, and if h was a *scrutinee* its plug fires CASERW, realizing a branch and pruning the others (§4.4). grounded(p) holds iff the demand tree of the realized path is empty.

4.4 The demand polynomial: `holes(·)` as a CDT

This section makes §2.4 precise: what `holes(p)` returns for an un-grounded, *branching* term, and why it is computed rather than annotated.

The object. A *conditional demand tree* is

$$\text{CDT} \;::=\; \langle\, \mathsf{now} : \mathsf{Frontier},\;\; \mathsf{wait} : \overline{\mathsf{Edge}} \,\rangle, \qquad \mathsf{Frontier} \;::=\; \overline{(h : T)},$$

$$\mathsf{Edge} \;::=\; \mathsf{on}(s)\,\{\, \overline{\kappa_i\, x_i \Rightarrow \text{CDT}_i} \,\},$$

where now is a finite set of holes armed *immediately and in parallel*; each Edge is a *subscription* on a scrutinee s (a neutral term stuck on an unresolved HOLE or OBS), with one continuation per constructor κ_i of s's variant type, binding the outcome payload x_i (so CDT_i and its hole-types may mention x_i). This is exactly a finite *dependent polynomial* / interaction structure: positions are outcome-states, the directions at a position are the holes owed there, the responses are the outcomes. now and wait at one node are concurrent — arm every now demand *and* subscribe to every edge at once.

Computing it (and why it halts). `holes(p)` is a two-step fold:

1. *Symbolic normalization.* Reduce p with agent holes held *opaque* (neutral). Strong normalization of grounded `PFLam` — preserved by finite variants (§10) — guarantees this terminates, yielding a *demand normal form*: a tree of CASEs on neutral scrutinees with hole-demands at the leaves.
2. *Extraction + hoist.* Read it structurally: a demand $?h{:}T$ not under any unresolved CASE joins now; a CASE on neutral s yields an $\mathsf{on}(s)\{\dots\}$ edge, recursively. Then iterate the *distributive law*

$$\mathsf{on}(s)\{\, \kappa_i \Rightarrow (\,\text{owe}(h{:}T) \parallel e_i\,)\,\}_i \quad\equiv\quad \text{owe}(h{:}T) \;\parallel\; \mathsf{on}(s)\{\, \kappa_i \Rightarrow e_i\,\}_i \qquad (\text{if } s, x_i \notin \text{fv}(T))$$

   to a fixpoint: a demand owed on *every* branch, and whose type does not mention the outcome, hoists to the parent frontier.

The side condition does the real work. It hoists h_{Cater} (owed on both branches, type outcome-independent) all the way to the root now, while h_{Tent} *cannot* hoist past $\mathsf{on}(v)$ because its type $\mathsf{Tent}(v_{\text{out}})$ binds the outcome. The *type dependency* — decidably checked — is precisely what fibers the demand. The fixpoint is canonical: each hole sits at the highest node at which its type is well-formed, i.e. it is armed as early as its dependencies permit and no earlier.

Spivak's example. For `alicePF` (§9),

$$\text{holes}(\texttt{alicePF}) = \Big\langle\, \mathsf{now} = \{\, h_{\text{Venue}} : O_{\text{venue}},\ h_{\text{Cater}} : \mathsf{Catering} \,\},\ \ \mathsf{wait} = [\, \mathsf{on}(h_{\text{Venue}})\, \mathcal{B}\,] \,\Big\rangle,$$

$$\mathcal{B} = \{\ \mathsf{Indoor}\ v_{\text{in}} \Rightarrow \langle \varnothing, [\,]\rangle,\quad \mathsf{Outdoor}\ v_{\text{out}} \Rightarrow \langle \{\, h_{\text{Tent}} : \mathsf{Tent}(v_{\text{out}}) \,\}, [\,]\rangle\ \}.$$

h_{Cater} and h_{Venue} are armed now, in parallel; h_{Tent} is owed *iff* h_{Venue} resolves Outdoor, with a type that depends on which outdoor venue (Figure 2).

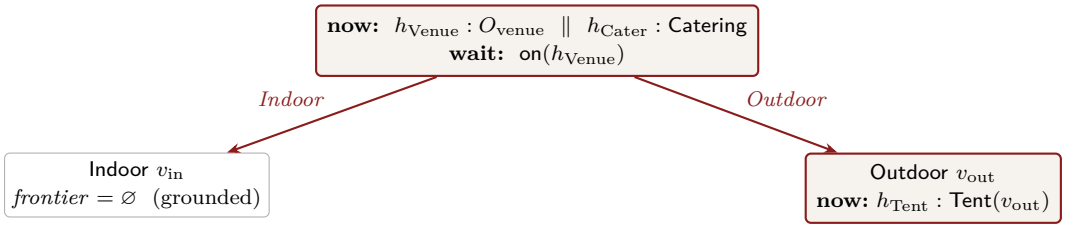

Figure 2: The CDT of `alicePF`. The root frontier (h_{Cater}, h_{Venue}) is armed at publish; the h_{Tent} demand exists in the term *now* (so the Outdoor branch can be priced and checked) but is *armed* only when h_{Venue} resolves Outdoor.

This is the third way out of the eager/lazy dilemma (§2.4): the contingency is *stated* in the term (so reasoning, pricing, and checking happen pre-grounding, recovering "lazy"'s loss), yet *armed* only on the realizing outcome (so indoor venues carry no spurious tent obligation, avoiding "eager"'s loss). When does a scrutinee resolve? If s is a filler-resolved HOLE whose type is a variant, plugging the filler reduces the CASE at *ground time* and the coordinator arms the sub-CDT during matching. If s is an OBS hole, the branch fires at *run time* on attestation. The CDT shape is identical; only the stage differs, and it is read off the hole's kind.

5 The bridge

MeTTaIL lets a defined language's terms be sent on rho channels. The coordination layer therefore carries `PFLam` terms as payloads and needs five host predicates/operators, each a rho fold that delegates to `PFLam`. We extend the embedded coordination language with:

```
// Bridge operators (rho-level folds delegating to PFLam). Surface syntax in
// the coordination layer; pf, ty, q are embedded PFLam terms carried as payload.

Models   . pf:Proc, ty:Proc  |- pf "|=" ty            : Proc ![{
    // decidable bidirectional check: PFLam.check(ctx_h, pf, ty)
    bool_proc(crate::pflam::models(&pf, &ty))
}] fold;

GroundedP. pf:Proc           |- "grounded" "(" pf ")" : Proc ![{
    bool_proc(crate::pflam::no_agent_holes(&pf))
}] fold;

PlugP    . pf:Proc, h:Proc, q:Proc
         |- "plug" "(" pf "," h "," q ")"             : Proc ![{
    crate::pflam::plug_hole_payload(&pf, &h, &q)      // → embedded PFLam term
}] fold;

// holes(pf) now returns a CONDITIONAL DEMAND TREE (sec. 4.4), not a list:
```

```
//   < now: [[h,ty]...], wait: [ on(s){ [kappa, contFn] ... } ... ] >
// where each cont is the sub-CDT as a function of the bound outcome payload.
HolesP   . pf:Proc            |- "holes" "(" pf ")"    : Proc ![{
    crate::pflam::demand_tree_payload(&pf)             // → CDT payload
}] fold;

// resolve(pf, s): the channel on which scrutinee s's outcome arrives. For a
// filler-resolved Hole it is derived from the plug (the filler's variant tag +
// payload); for an Obs hole it is the run-time attestation channel. Lets the
// coordinator SUBSCRIBE to a branch edge and continue with the right sub-CDT.
Resolve  . pf:Proc, s:Proc    |- "resolve" "(" pf "," s ")" : Proc ![{
    crate::pflam::resolve_channel(&pf, &s)             // → Name (a rho channel)
}] fold;
```

With these in scope, the matching guard Spivak's framing asks for is literally a rholang `where` clause: `where pf= ty|`. Everything in §6 is ordinary embedded rholang plus these operators: `now`-frontier holes become guarded demands, and `wait`-edges become `resolve`-subscriptions.

6 Coordination protocol (embedded rholang)

6.1 Namespaces and channel conventions

To each user (say `alice`) we associate a namespace of channels, addressed by *quoting* structured names. We write channels as quoted lists; e.g. `@[alice, "demand", h]` is the demand channel for hole `h` of one of Alice's PFs. Conventions:

Channel	Role
`@[u, "pf", pfId]`	canonical store of user u's PF *pfId* (latest term)
`@[u, "demand", pfId, h]`	open agent hole h: candidates are offered here
`@[u, "resolve", pfId, s]`	outcome of scrutinee s (filler tag at ground time, or attestation)
`@[u, "supply"]`	u publishes grounded/partial PFs it offers to others
`@[u, "prov", pfId]`	provenance map: hole $\mapsto$ (party, fillerId)
`@[u, "ground", pfId]`	fires once *pfId* becomes grounded
`@[u, "run", pfId]`	execution controller for a grounded *pfId*
`@[u, "step", pfId, k]`	attestation channel for execution step k
`@["mkt", "templates"]`	public registry of successful PFs (templates)
`@["mkt", "caveats"]`	public registry of advertised failures (anti-templates)

6.2 Publish: Draft → Published

Publishing stores the PF and walks its *conditional demand tree* (§4.4): it arms one persistent, type-guarded demand for every hole in the current `now`-frontier, and *subscribes* to every `wait`-edge so that a sub-CDT is armed only when its scrutinee resolves. Each demand is the generalization of Spivak's `for( pf <- aliceChan where pf= type ) aliceChan!( (alicePF pf) ) |`; the subscription is the new ingredient that makes branch-dependent holes (the tent) behave correctly.

```
// publish(u, pfId, pf): store, then arm the CDT of pf.
new publish, armCDT in {
  contract publish(@u, @pfId, @pf) = {
    @[u, "pf", pfId]!!( pf )                          // canonical store (persistent)
```

```
    | for( @cdt <- holes(pf) ) { armCDT!( u, pfId, cdt ) }
  }
  |
  // armCDT walks one CDT node: arm every now-hole in PARALLEL, and subscribe to
  // every wait-edge so sub-CDTs are armed only on the realizing outcome.
  contract armCDT(@u, @pfId, @cdt) = {
    match cdt {
      < now: [], wait: [] >      => @[u, "ground", pfId]!( pf )   // realized path empty
      < now: ns,  wait: ws >     => {
        // (1) frontier: one guarded persistent demand per now-hole, all concurrent
        for( @[h, ty] <- ns ) { armDemand!( u, pfId, h, ty ) }
        // (2) edges: subscribe to each scrutinee; arm the matching sub-CDT on resolve
        | for( @on(s, branches) <- ws ) {
            for( @out <- resolve(pf, s) ) {            // outcome arrives (ground- or run-time)
              match out {
                // pick the branch for the realized constructor, binding its payload x,
                // and arm that sub-CDT (its hole-types may mention x).
                @[kappa, x] => armCDT!( u, pfId, select(branches, kappa, x) )
              }
            }
          }
      }
    }
  }
  |
  // one guarded, persistent demand for hole h with expected type ty
  contract armDemand(@u, @pfId, @h, @ty) = {
    for( @cand <= @[u, "demand", pfId, h] where cand |= ty ) {   // type mediates matching
      for( @pf <- @[u, "pf", pfId] ) {                          // read current term
        new pf2 in {
          pf2!( plug(pf, h, cand) ) |                            // fill the hole
          for( @next <- pf2 ) {
            recordProvenance!( u, pfId, h, cand ) |
            // if h was a scrutinee, plug fired CaseRw: the branch is now realized,
            // so re-deriving holes(next) yields the correct residual CDT.
            republish!( u, pfId, next )
          }
        }
      }
    }
  }
}
```

Notes. (i) The frontier holes are armed *concurrently* — `hCater` and `hVenue` race freely; neither waits on the other. (ii) A `wait`-edge arms nothing until `resolve(pf,s)` delivers an outcome, so the *indoor* path never arms `hTent`. (iii) `resolve` unifies the two stages: for a filler-resolved scrutinee it fires when the plug commits a variant (ground time); for an `Obs` scrutinee it fires on attestation (run time). (iv) `republish` re-enters *publish*, re-deriving the CDT of the new term — this is where a filler's own holes, and any branch just realized, become the next frontier.

6.3 Offer: supply side

Bob offers a PF by sending it where a broker (or Alice directly) can pick it up. In the minimal form this is Spivak's `bobChan!( bobPF )`:

```
// Bob advertises a (possibly still-holed) PF he is willing to provide.
@[bob, "supply"]!!( bobPF )
```

6.4 Broker: forwarders with optional type filtering

A broker connects supply to demand by creating forwarders. The bare forwarder is Spivak's `aliceChan!( pf ) for( pf <- bobChan )` ; we add the type guard so the broker only forwards plausibly-matching candidates and does not spam Alice's demand:

```
// broker(bob → alice@(pfId,h:ty)): forward Bob's supply to Alice's demand,
// pre-filtered by the hole type so only viable candidates flow.
contract broker(@bob, @alice, @pfId, @h, @ty) = {
  for( @cand <= @[bob, "supply"] where cand |= ty ) {
    @[alice, "demand", pfId, h]!( cand )      // Alice's guarded demand re-checks and plugs
  }
}
```

The double check (broker filters, demand re-checks) is intentional: the broker's filter is an optimization; the demand's guard is the authority. Brokers may be matchmakers, reputation services, or auctions; none of them can cause an ill-typed plug, because the plug only happens behind Alice's guard.

6.5 Partial grounding and recursion

Each successful plug transforms the demand tree, not merely a flat set: `republish` re-derives `holes(next)`, which re-arms the residual `now`-frontier (old minus h, plus q's own frontier) and re-subscribes to the residual edges. If h was a scrutinee, its plug realized a branch, so the residual CDT is the corresponding fiber — the untaken branches and their holes are gone. The market thus explores a tree of sub-PFs of unbounded depth, each node owned by whichever participant supplied it. Termination of *a particular* PF is exactly: every realized path eventually reaches grounded fillers — a liveness property of the market, not a theorem (§10).

6.6 Grounding detection: → Grounded

Grounding is now *path-realized*, not set-empty. As branches resolve, CASERW prunes the untaken arms, and `holes(pf)` of the residual term shrinks toward `< now: [], wait: [] >`. When `armCDT` reaches that node — the realized path owes nothing and waits on nothing — it emits on `@[u,"ground",pfId]`. That event is the precondition for execution. (A PF with an unresolved `Obs` edge is *not* grounded: it still waits on a run-time outcome, so its tail is executed under monitoring rather than dispatched up front — §6.7.)

6.7 Execution: Grounded → Running

Running a grounded PF means: walk its plan, dispatch each leaf obligation to the party who supplied it (read from provenance), and collect attestations. Because the obligations are discharged *in the world*, attestations are signed acknowledgements on per-step channels (oracles); rho coordinates and records, it does not perform the physical work.

```
// run(u, pfId): dispatch obligations and start the monitor.
contract run(@u, @pfId) = {
  for( @pf <- @[u, "ground", pfId] ; @prov <- @[u, "prov", pfId] ) {
    new steps in {
      steps!( plan(pf, prov) ) |                // ordered/partially-ordered obligations
      for( @plan <- steps ) {
        dispatch!( u, pfId, plan ) |            // notify each party of its step
        monitor!( u, pfId, plan, 0 )            // begin monitoring at step 0
```

```
      }
    }
  }
}

// dispatch: tell each contributing party what to do, on their own channel.
contract dispatch(@u, @pfId, @plan) = {
  for( @[party, k, obligation] <= @plan ) {      // stream the plan's leaves
    @[party, "todo", pfId, k]!( obligation )     // party performs, then attests on step k
  }
}
```

6.8 Monitoring, barriers, and coordination

The monitor advances only on positive attestation, joins steps that must complete together (a *barrier* via multi-receive `&`), and routes timeouts/negative acks to failure.

```
// monitor advances step-by-step; a barrier step waits on several attestations at once.
contract monitor(@u, @pfId, @plan, @k) = {
  for( @stepK <- stepAt(plan, k) ) {
    match stepK {
      // ordinary step: one attestation
      [party, k, _] =>
        for( @ack <- @[u, "step", pfId, k] where ack == "ok" ) {
          advance!( u, pfId, plan, k )
        }
      // barrier step: all listed parties must attest before advancing
      [barrier, ks] =>
        for( @a <- @[u,"step",pfId, nth(ks,0)] where a == "ok"
           & @b <- @[u,"step",pfId, nth(ks,1)] where b == "ok" ) {     // ... & ... per ks
          advance!( u, pfId, plan, k )
        }
    }
  }
  // failure routing (timeout or negative attestation) handled in parallel:
  | for( @ack <- @[u, "step", pfId, k] where ack != "ok" ) {
      fail!( u, pfId, pfId )                      // → retirement (sec. 6.11)
    }
}

// advance: last step → success; otherwise continue.
contract advance(@u, @pfId, @plan, @k) = {
  match isLast(plan, k) {
    true  => succeed!( u, pfId )
    false => monitor!( u, pfId, plan, k + 1 )
  }
}
```

On timeouts. Pure rho has no clock; "timeout" is supplied by an external timer oracle that posts a non-`"ok"` attestation on the step channel after a deadline. The monitor treats it uniformly as a failed step.

6.9 Success → Template

If every step attests `"ok"`, the PF *and its sub-PFs* become reusable templates. Templating is *re-abstraction*: take the grounded composite and reintroduce holes at the positions that were filled (per provenance), yielding a parametric PF that can be re-grounded with different fillers later. The principled version of "reintroduce holes at the filled positions" is anti-unification (least general generalization) over the filled subterms (§10).

```
contract succeed(@u, @pfId) = {
  for( @pf <- @[u, "pf", pfId] ; @prov <- @[u, "prov", pfId] ) {
    new tmpl in {
      tmpl!( abstract(pf, prov) ) |              // re-introduce holes at filled positions
      for( @t <- tmpl ) {
        @["mkt", "templates"]!!( [u, pfId, t] )   // publish reusable template
        | @[u, "supply"]!!( t )                    // u can now offer the pattern to others
      }
    }
  }
}
```

6.10 Failure → Retirement: hide vs. advertise

Two retirements, per the brief. Both first *withdraw* the PF's live demands (consume the persistent demand receivers so no further candidates are accepted). They differ in what they make public.

```
contract fail(@u, @pfId, @reason) = {
  withdrawDemands!( u, pfId ) |                   // stop all guarded demands for this PF
  // caller chooses one of the two retirements:
  Nil
}

// (a) HIDE: keep privately, advertise nothing. Quiet failure.
contract hide(@u, @pfId) = {
  for( @pf <- @[u, "pf", pfId] ) {
    @[u, "hidden", pfId]!!( pf )                  // retained privately; absent from market
  }
}

// (b) ADVERTISE: broadcast the failed pattern so the market routes around it.
//     An anti-template / caveat that other agents can match against.
contract advertise(@u, @pfId, @reason) = {
  for( @pf <- @[u, "pf", pfId] ) {
    @["mkt", "caveats"]!!( [u, pfId, abstract(pf, reason)] )   // public anti-template
  }
}
```

Symmetry worth noting: success publishes to `@["mkt","templates"]` ("do this"); loud failure publishes to `@["mkt","caveats"]` ("this didn't work"). Both feed future matching — templates as supply, caveats as filters.

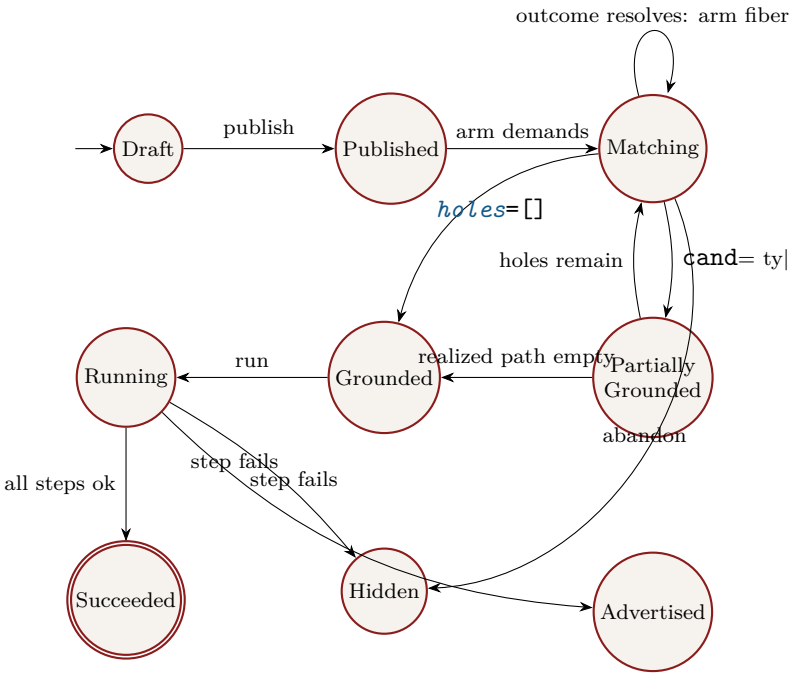

Figure 3: PF lifecycle. *Matching* ↔ *Partially Grounded* is the recursive core; *Succeeded* promotes to a template; failure retires to *Hidden* (quiet) or *Advertised* (loud).

7 Lifecycle state machine

From	To	Guard / event	Coordination effect
Draft	Published	*publish*	store PF; compute CDT
Published	Matching	now ≠ ∅ or wait ≠ ∅	arm frontier; subscribe to edges
Matching	Part. Grounded	cand= ty|	*plug*; record provenance; republish
Matching	Matching	scrutinee resolves κ	arm the κ-fiber's sub-CDT
Part. Grounded	Matching	residual CDT non-empty	re-arm frontier (incl. filler's holes)
Matching / Part.	Grounded	realized path empty	emit @[u,"ground",pfId]
Grounded	Running	run	plan; dispatch obligations; start monitor
Running	Succeeded	all steps "ok"	*abstract*; publish template
Running	Hidden	step fails (quiet)	withdraw demands; retain privately
Running	Advertised	step fails (loud)	withdraw demands; publish caveat
Matching	Hidden	abandon	withdraw demands

8 Requirements catalogue

Functional

FR-1 A PF is a PFLam term with typed agent holes; the system stores each PF on a canonical per-user channel and treats that channel as the single source of truth for its current term.

FR-2 The system shall distinguish agent holes from computation holes; only agent holes generate

market demands, and grounded shall count only agent holes.

FR-3 `holes(pf)` shall return a *conditional demand tree* (CDT): a `now`-frontier of holes plus `wait`-edges over unresolved scrutinees, computed by symbolic normalization and the hoist distributive law (§4.4). It shall not return a flat list.

FR-3a For each hole in the current `now`-frontier the system shall arm a persistent, type-guarded demand that accepts a candidate iff `cand`= ty|; all frontier demands are armed concurrently.

FR-3b For each `wait`-edge the system shall subscribe to the scrutinee's resolution and arm the matching branch's sub-CDT *only* when that outcome lands. A branch hole shall never be armed on a path that does not realize its branch (no spurious obligations).

FR-4 Acceptance shall *plug* the candidate, record provenance (hole ↦ party, filler), and republish the resulting term, re-deriving its CDT — including any branch just realized (CaseRw) and any holes the filler itself introduces.

FR-5 Participants shall be able to offer PFs (supply) and brokers shall be able to create type-filtered forwarders from supply to demand; brokers shall not be able to cause an ill-typed plug.

FR-6 When the realized path's residual CDT is empty (`now=[]`, `wait=[]`) the system shall mark the PF Grounded and make it eligible to run. A PF still waiting on an `Obs` (run-time) scrutinee is not Grounded.

FR-7 Running shall dispatch each leaf obligation to the supplying party (from provenance) and begin monitoring.

FR-8 Monitoring shall advance only on positive attestation, shall support barrier steps via atomic multi-receive, and shall route negative/late attestations to failure.

FR-9 On full success the system shall re-abstract the composite into a template and publish it to the public template registry and the owner's supply channel.

FR-10 On failure the system shall withdraw the PF's live demands and perform exactly one of two retirements: *hide* (retain privately, publish nothing) or *advertise* (publish an anti-template/caveat to the public registry).

FR-11 Templates shall be re-groundable: a published template is itself a PF with holes and re-enters the lifecycle when adopted.

Non-functional

NFR-1 *Decidable acceptance.* The matching guard =| must terminate on the supported `PFLam` fragment (normalizing; conversion decidable).

NFR-2 *No solver in the loop.* The system must not attempt to synthesize fillers; it only checks offered candidates (§2.5).

NFR-3 *Capability discipline.* Channel names are unforgeable quoted names; a party can act on a hole/step only if it holds the corresponding name.

NFR-4 *Attested execution.* World-effecting steps are confirmed only by signed oracle attestations; the coordinator never asserts physical completion itself.

NFR-5 *Auditability.* Provenance, grounding, attestations, templates, and caveats are all on-channel and replayable.

NFR-6 *Monotone learning.* Templates and caveats accumulate and feed future matching without mutating past records.

NFR-7 *Branching preserves termination.* The variant fragment (finite sums + dependent eliminator, no recursion) shall preserve strong normalization, so `holes` and the matching guard remain terminating (§10).

NFR-8 *Canonical demand placement.* The CDT shall be hoist-normal: every hole sits at the highest node at which its type is well-formed, i.e. armed as early as its dependencies permit and no earlier (§4.4).

9 Worked example

We use Spivak's branching case directly. Alice books a venue before she knows whether it will be *indoor* or *outdoor*; catering is owed either way, but a tent (rain contingency) is owed *iff* the venue comes back outdoor, and the tent's type depends on *which* outdoor venue. This is the minimal term whose open-hole set is a function of an outcome not yet observed.

```
// Alice's PF (PFLam term). hCater is flat; hVenue is a scrutinee whose variant
// outcome decides whether hTent is owed. O_venue = < Indoor : V_in | Outdoor : V_out >.
alicePF =
  reqs .                                          // root: date, headcount, budget
  call_cater( reqs ; c .                          // c : Catering        -- owed on EVERY branch
  case (? hVenue : O_venue) of {                  // scrutinee = a filler-resolved hole
    Indoor  v_in  => ret (booking v_in  c)            // indoor: venue + catering
    Outdoor v_out => ret (booking v_out c             // outdoor: venue + catering + TENT
                          (? hTent : Tent v_out))     // hTent's TYPE depends on v_out
  })

// holes(alicePF)  -- the CDT (sec. 4.4), NOT a flat list:
//   < now:  [ [hVenue, O_venue], [hCater, Catering] ]          // armed concurrently
//   , wait: [ on(hVenue) {
//               Indoor  v_in  => < now: [],                      wait: [] >    // grounded
//               Outdoor v_out => < now: [[hTent, Tent v_out]],   wait: [] >    // tent owed here
//             } ] >
//   hCater hoisted to root (owed on both branches, type outcome-independent);
//   hTent pinned under Outdoor (its type binds v_out, so it cannot hoist).

// 1. PUBLISH: arm frontier {hVenue, hCater} now; subscribe to on(hVenue).
publish!( alice, "event", alicePF )

// 2. OFFERS + BROKER (frontier holes only; NO tent demand exists yet)
@[bob,   "supply"]!!( bobOutdoorVenue )   // bobOutdoorVenue |= O_venue  (an Outdoor v_out)
@[carol, "supply"]!!( carolCater )        // carolCater      |= Catering
broker!( bob,   alice, "event", "hVenue", O_venue )
broker!( carol, alice, "event", "hCater", Catering )

// 3. hCater and hVenue match CONCURRENTLY behind Alice's guards; each plugs+republishes.
//    Plugging hVenue with an Outdoor filler fires CaseRw → the Outdoor branch is realized:
//    resolve(pf, hVenue) delivers [Outdoor, v_out], so armCDT now arms the fiber:
//      armDemand!( alice, "event", "hTent", Tent v_out )   // tent demanded ONLY now
//    (had bob supplied an Indoor venue, resolve delivers [Indoor, v_in]; hTent never arms.)
```

```
// 4. TENT offer for the now-armed, outcome-dependent demand
@[dave, "supply"]!!( daveTent )           // daveTent |= Tent v_out   (depends on the venue!)
broker!( dave, alice, "event", "hTent", Tent v_out )
//    plug hTent → residual CDT = < now: [], wait: [] >  → GROUND

// 5. RUN: dispatch obligations from provenance (venue, catering, tent)
//    @[bob,"todo","event",0]!(...) | @[carol,...1]!(...) | @[dave,...2]!(...)

// 6a. SUCCESS: all attest "ok" → abstract → @["mkt","templates"]!!(["alice","event",t])
// 6b. FAILURE: e.g. tent step fails → withdrawDemands ; then
//        hide!( alice, "event" )                    // quiet, or
//        advertise!( alice, "event", "tent-failed" ) // public caveat
```

The example exhibits every moving part of Option A at once. *Concurrency:* `hCater` and `hVenue` are armed together — catering does not wait on the venue decision. *Fibered demand:* `hTent` is present in the term from the start (so the outdoor branch can be priced and checked before anything grounds), yet it is *armed* only after `hVenue` resolves `Outdoor`; an indoor outcome arms no tent, so the spurious-obligation problem never arises. *Dependency does the pinning:* `hTent`'s type `Tent v_out` binds the outcome payload, so the hoist law cannot lift it above `on(hVenue)` — the type system, decidably, is what fibers the demand. *Ground- vs run-time* is one line's difference: written `? hVenue` the branch resolves at ground time (the chosen venue's variant tag); written `observe hVenue` (an OBS hole) the same CDT resolves at run time, on attestation — which is the shape Alice will want for an outcome like "opening-day turnout" that only execution reveals.

10 Established vs. conjectural; open problems

We separate what the design *relies on and gets* from what it *hopes for*.

Established (given a normalizing fragment).

- *Acceptance is decidable.* Checking $q \Leftarrow T$ (hence `q=` ty|) is decidable when conversion is decidable; bidirectional checking against a *given* candidate needs no higher-order unification.
- *Plugging is total and structural;* on the flat fragment the hole-set algebra $\text{holes}(p \triangleleft_h q) = (\text{holes}(p) \setminus \{h\}) \cup \text{holes}(q)$ holds definitionally, and with branching it is the corresponding splice on the demand tree.
- *Branching preserves strong normalization.* Adding finite (dependent) variants with their eliminator and commuting conversions, *without recursion*, is the standard SN-conservative extension (Tait/Girard reducibility extends to finite coproducts). Hence `holes` terminates and the CDT is finite. The only SN risk is recursion/fixpoints, which this fragment excludes (NFR-7).
- *The CDT is well-defined and canonical.* Symbolic normalization yields a unique demand normal form, and the hoist law has a fixpoint placing each hole at the highest type-admissible node (NFR-8); the dependency side condition is a decidable free-variable check.
- *The coordination layer is ordinary rho;* matching, brokering, joins/barriers, persistent demands, and the frontier-vs-subscription split are exactly the embedded-rholang constructs MeTTaIL already provides, plus the five bridge folds.

Conjectural / delegated / out of scope for the calculus.

- *Filler search is undecidable in general.* Synthesizing a q with $q \models T$ is higher-order search; we deliberately delegate it to the market (§2.5). "Every PF eventually grounds" is a market *liveness* property, not a theorem; it depends on participation and incentives.
- *Full dependent types may not normalize / convert decidably* once one adds the features a real marketplace wants (large elimination, general recursion, effects). The supported fragment must be fixed deliberately; NFR-1/NFR-7 constrain that choice, trading expressiveness for a decidable $=|$ and a finite CDT.
- *Staging of branch resolution.* Whether a scrutinee resolves at ground time (a filler-resolved HOLE whose plug commits a variant) or run time (an OBS hole resolved by attestation) is a real modelling decision per branch, not a default. The CDT is identical; the stage is read off the hole kind. Which outcomes are knowable pre-grounding vs only by execution is domain-specific and must be chosen with the user.
- *Valuation is the next layer, and it does not live in the type.* "Earlier is better", "more customers on opening day is better" are preferences over the *maximal plays* of the CDT, not predicates and not types. The natural object is a grading $val : \mathrm{Runs}(\mathrm{CDT}) \to Q$ valued in an ordered monoid / preference quantale Q, composed along the tree and used to *rank* admissible offers and to price branches against an outcome distribution. This is the same quantale-grading shape as phlogiston cost, valued in preference rather than resource — a deliberate continuity with the cost-accounted line of work, and out of scope for this document beyond naming it.
- *Templating as anti-unification.* "Reintroduce holes at filled positions" is clean for first-order positions; the principled general construction is least general generalization, which for typed/higher-order terms is subtle (no unique lgg in general). `abstract` is specified behaviourally here; its exact generalization order is open. (Branching makes this richer: a template may abstract over *which branch* was taken, i.e. generalize a realized path back to the polynomial.)
- *Execution is off-chain.* "All parties do their part in the external world" cannot be enforced by rho; NFR-4 reduces it to attestations from trusted oracles/timers. The trust model for those oracles, and the dispute resolution when attestations conflict, are out of scope here and should be designed next (this is also the natural seam for an economic/escrow layer).
- *Matching beyond checking.* If we later want the system to *propose* partial fillers (not just accept offered ones), we re-enter higher-order unification; a pragmatic middle path is Miller-pattern unification, decidable on the pattern fragment.

Suggested next decisions. (1) Fix the `PFLam` fragment, its conversion algorithm, and the precise variant/eliminator rules (motive discipline). (2) Specify `plan` and `abstract` precisely (ordering of obligations; generalization order, including over branches). (3) Decide, per outcome, ground-time vs run-time resolution (HOLE vs OBS). (4) Design the valuation layer: choose Q, the play-payoff, and how the market consumes it to rank offers and price branches. (5) Define the oracle/attestation trust model and where escrow attaches. (6) Decide whether brokers are privileged matchmakers, open auctions, or both, and what reputation signal (templates vs. caveats) they consume.

Glossary

PF	Plausible Fiction: a `PFLam` term with typed agent holes.
agent hole	A hole filled by another participant's PF; a market demand.

CDT Conditional demand tree: the value of *holes*(pf) — a now-frontier plus wait-edges over unresolved scrutinees; a finite dependent polynomial.

frontier The holes at a CDT node armed immediately and in parallel (now).

scrutinee A neutral term (stuck on an unresolved HOLE/OBS) a branch waits on.

observation hole An agent hole (OBS) resolved by a run-time attestation, not a plug.

computation hole A redex/elaboration metavariable; filled by reduction, never advertised.

hoist law The distributive rule lifting a branch-uniform, outcome-independent demand to the parent frontier; iterated to a canonical fixpoint.

grounded The realized path's residual CDT is empty; the PF is runnable.

plug $p \triangleleft_h q$: fill agent hole h in p with filler q (capture-avoiding).

|= *models/satisfies*: q= ty| iff q checks at type ty; the matching guard.

provenance Map hole $\mapsto$ (party, fillerId), used to dispatch obligations and to abstract.

template A re-abstracted, reusable PF promoted from a success.

caveat A publicly advertised failed pattern (anti-template).

valuation Future layer: a grading $val : \text{Runs(CDT)} \to Q$ ranking outcomes by preference (not a type, not a predicate).